MODUL 15 · BAB V · ALGORITMA GREEDY

Kode Huffman: Kompresi Data dengan Algoritma Greedy

Memberi kode pendek untuk karakter yang sering muncul dan kode panjang untuk yang jarang, tanpa kehilangan kemampuan decode.

TUJUAN PEMBELAJARAN

Setelah mempelajari modul ini, kamu diharapkan mampu:

- Menghitung ukuran data dengan kode panjang tetap (*fixed-length*) dan panjang berubah (*variable-length*).
- Menjelaskan kode prefiks dan alasan kode prefiks selalu dapat di-decode tanpa ambigu.
- Menjalankan algoritma HUFFMAN langkah demi langkah dan membentuk pohon serta kode Huffman.
- Melakukan encoding dan decoding dengan kode Huffman.
- Menjelaskan pilihan greedy pada Huffman dan running time $O(n \lg n)$.

PRASYARAT

- Modul 13: konsep Greedy.
- Pengetahuan dasar bilangan biner dan pohon biner.

BAB / MATERI ASAL

Materi 10.2 ALGORITMA GREEDY-ALGORITMA HUFFMAN

POSISI DALAM URUTAN BELAJAR

Penerapan kedua Greedy (setelah Modul 14). Menutup Bab V.

DAFTAR ISI

1. Mengapa Setiap Karakter Harus 8 Bit? 1
2. Kode Panjang Tetap dan Panjang Berubah 2
3. Kode Prefiks 2
4. Algoritma HUFFMAN 3
5. Membangun Pohon Huffman untuk Contoh 4
6. Kebenaran dan Running Time 6
7. Kesimpulan 7

◆ Kuis Pemahaman 9
◆ Kunci Jawaban dan Pembahasan 10
◆ Referensi 10

PENDAHULUAN

1. Mengapa Setiap Karakter Harus 8 Bit?

Pada pengkodean ASCII, setiap karakter disimpan dalam 8 bit (1 byte). Misalnya huruf `A` memiliki nilai desimal 65, yang dalam biner ditulis `01000001`. Cara ini sederhana, tetapi boros: huruf yang sangat sering muncul memakai bit sebanyak huruf yang hampir tidak pernah muncul.

Kompresi data adalah teknik untuk memperkecil ukuran data dari ukuran aslinya, sehingga ruang penyimpanan (memori) dan waktu pengiriman dapat dihemat. Salah satu gagasan dasarnya: beri **kode**

pendek untuk karakter yang **sering** muncul dan **kode panjang** untuk karakter yang **jarang** muncul, seperti sandi Morse yang memberi huruf E kode paling pendek.

David A. Huffman (1952) menemukan cara membangun kode semacam itu yang **optimal** dengan algoritma Greedy yang sangat sederhana. Modul ini membahas konsep kode panjang tetap dan panjang berubah, kode prefiks, algoritma HUFFMAN, serta cara melakukan encoding dan decoding.

PEMBAHASAN INTI

2. Kode Panjang Tetap dan Panjang Berubah

Contoh dari bahan kuliah: sebuah berkas berisi **100.000 karakter** yang terdiri atas 6 jenis karakter dengan frekuensi kemunculan sebagai berikut.

Karakter	a	b	c	d	e	f
Frekuensi (ribuan)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101
Variable-length codeword	0	101	100	111	1101	1100

Tabel 1. Frekuensi dan dua cara pengkodean.

Cara	Perhitungan	Total bit
Tanpa kompresi (ASCII 8 bit)	100.000×8	800.000
Kode panjang tetap	6 jenis karakter butuh minimal 3 bit ($2^2 = 4 < 6 \leq 8 = 2^3$), jadi 100.000×3	300.000
Kode panjang berubah	$(45 \cdot 1 + 13 \cdot 3 + 12 \cdot 3 + 16 \cdot 3 + 9 \cdot 4 + 5 \cdot 4) \times 1.000$	224.000

Tabel 2. Ukuran berkas dengan tiga cara penyimpanan.

Penghematan kode panjang berubah dibandingkan kode panjang tetap:

$$\frac{300.000 - 224.000}{300.000} \times 100\% \approx 25{,}3\%$$

(bahan kuliah membulatkannya menjadi 25%). Dibandingkan ASCII 8 bit, penghematannya 72%.

PEMBAHASAN INTI

3. Kode Prefiks

Kode panjang berubah memunculkan masalah baru: bagaimana mengetahui di mana satu kode berakhir dan kode berikutnya dimulai? Perhatikan kode buruk berikut: `a = 0`, `b = 01`, `c = 1`. Barisan bit `01` bisa dibaca sebagai "ac" atau "b". Kode seperti ini **ambigu** dan tidak dapat di-decode dengan pasti.

DEFINISI

Kode prefiks (*prefix code*) adalah kode yang tidak satu pun kodenya menjadi **awalan (prefiks)** dari kode karakter lain.

Pada kode panjang berubah di atas:

- `a = 0` (1 bit). Tidak ada kode lain (b, c, d, e, f) yang diawali `0`.
- `b = 101` (3 bit). Kode 4 bit (e dan f) tidak ada yang diawali `101`; demikian pula untuk `100` dan `111`.

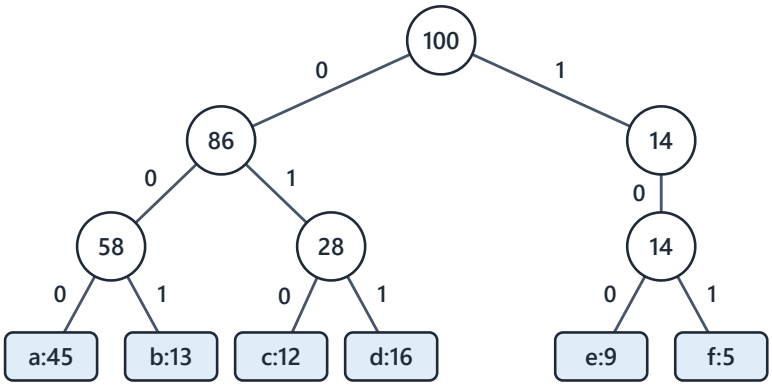
Karena itu kode Huffman **pasti dapat di-decode**: baca bit dari kiri, dan begitu barisan bit yang terbaca cocok dengan suatu kode, itulah karakternya, karena tidak mungkin ada kode lain yang lebih panjang dengan awalan yang sama.

3.1 Kode prefiks sebagai pohon biner

Kode prefiks dapat digambarkan sebagai **pohon biner**: setiap karakter adalah daun, busur ke anak kiri diberi label 0, busur ke anak kanan diberi label 1, dan kode suatu karakter adalah barisan label dari akar ke daunnya. Panjang kode = kedalaman daun. Total bit untuk berkas adalah

$$B(T) = \sum_{c \in C} c.freq \cdot d_T(c),$$

dengan $d_T(c)$ adalah kedalaman daun c pada pohon T. Kode yang optimal selalu berupa **pohon biner penuh** (setiap simpul dalam memiliki tepat dua anak); simpul dalam yang hanya punya satu anak berarti ada bit yang terbuang (Cormen dkk., 2009).



Gambar 1. Pohon kode panjang tetap (3 bit). Simpul '14' di kanan hanya punya satu anak, tanda bahwa kode ini tidak optimal.

PEMBAHASAN INTI

4. Algoritma HUFFMAN

Huffman membangun pohon kode secara **bottom-up**: dimulai dari $|C|$ daun (satu untuk setiap karakter), lalu berulang kali **menggabungkan dua simpul dengan frekuensi terkecil** menjadi satu simpul baru yang frekuensinya adalah jumlah keduanya. Setelah $|C| - 1$ kali penggabungan, tersisa satu pohon.

HUFFMAN(C)

```
1  n = |C|
2  Q = C
3  for i = 1 to n - 1
4      allocate a new node z
5      z.left = x = EXTRACT-MIN(Q)
6      z.right = y = EXTRACT-MIN(Q)
7      z.freq = x.freq + y.freq
8      INSERT(Q, z)
9  return EXTRACT-MIN(Q)          // akar pohon
```

- C: himpunan karakter beserta frekuensi kemunculannya; $|C|$: banyaknya karakter.
- Q: antrean prioritas minimum (*min-priority queue*) yang terurut berdasarkan frekuensi.
- `EXTRACT-MIN(Q)`: mengeluarkan simpul dengan frekuensi terkecil dari Q.
- `INSERT(Q, z)`: memasukkan simpul z ke Q sesuai urutan frekuensi.
- Langkah 4–8: buat simpul baru z; anak kirinya simpul terkecil x, anak kanannya simpul terkecil berikutnya y; frekuensi z = jumlah keduanya; masukkan z kembali ke Q.

PILIHAN GREEDY

Di setiap langkah, Huffman menggabungkan dua simpul dengan frekuensi **terkecil**. Alasannya: karakter yang paling jarang muncul sebaiknya berada paling dalam (kodenya paling panjang), karena bit tambahan pada karakter yang jarang paling sedikit menambah total bit.

CONTOH DAN PENERAPAN

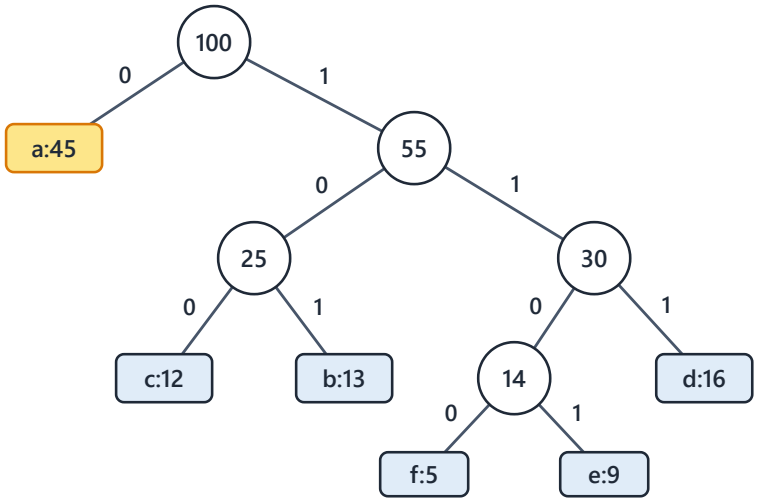
5. Membangun Pohon Huffman untuk Contoh

5.1 Isi antrean Q pada setiap langkah

Tahap	Gabungkan	Isi Q (terurut menurut frekuensi)
Langkah 2 (awal)	-	f:5, e:9, c:12, b:13, d:16, a:45
Loop 1 ($i = 1$)	f:5 + e:9 = 14	c:12, b:13, [14], d:16, a:45
Loop 2 ($i = 2$)	c:12 + b:13 = 25	[14], d:16, [25], a:45
Loop 3 ($i = 3$)	[14] + d:16 = 30	[25], [30], a:45
Loop 4 ($i = 4$)	[25] + [30] = 55	a:45, [55]
Loop 5 ($i = 5$)	a:45 + [55] = 100	[100] (akar)

Tabel 3. Isi Q setelah setiap putaran (simpul gabungan ditulis dalam kurung siku).

Pada loop 1, simpul 14 disisipkan di antara b:13 dan d:16. Pada loop 2, simpul 25 disisipkan di antara d:16 dan a:45. Pada loop 3, simpul 30 ditempatkan di antara simpul 25 dan a:45. Pada loop 4, simpul 55 ditempatkan setelah a:45, karena $45 < 55$. (Bahan kuliah menulis "disisipkan antara karakter a dan simpul z = 55"; maksudnya simpul 55 berada setelah a:45.)



Gambar 2. Pohon Huffman hasil akhir. Label 0 untuk busur ke anak kiri, 1 untuk busur ke anak kanan.

5.2 Kode Huffman yang dihasilkan

Karakter	Frekuensi	Jalur dari akar	Kode	Panjang	Frekuensi × panjang
a	45	kiri	0	1	45
b	13	kanan, kiri, kanan	101	3	39
c	12	kanan, kiri, kiri	100	3	36
d	16	kanan, kanan, kanan	111	3	48
e	9	kanan, kanan, kiri, kanan	1101	4	36
f	5	kanan, kanan, kiri, kiri	1100	4	20
Total (ribuan bit)					**224**

Tabel 4. Kode Huffman dari pohon di atas.

Kode ini persis kode panjang berubah pada tabel awal, dan karakter paling sering (a) mendapat kode terpendek, sedangkan yang paling jarang (e, f) mendapat kode terpanjang.

5.3 Encoding dan decoding

Encoding kata "face": f = `1100`, a = `0`, c = `100`, e = `1101`, sehingga hasilnya `110001001101` (12 bit, dibandingkan 32 bit dengan ASCII).

Decoding barisan `10111011000` dari bahan kuliah. Baca bit dari kiri dan ikuti pohon dari akar; setiap kali tiba di daun, catat karakternya dan kembali ke akar:

Bit yang dibaca	101	1101	100	0
Karakter	b	e	c	a

Tabel 5. Decoding 10111011000.

Hasilnya kata **"beca"**. Tidak ada keraguan di setiap langkah, karena kodenya kode prefiks.

5.4 Implementasi Python

Antrean prioritas diwujudkan dengan *min-heap* (`heapq`). Sebuah pencacah ditambahkan sebagai pemecah seri agar urutan karakter yang frekuensinya sama selalu konsisten.

Python: HUFFMAN, encoding, dan decoding

```python
import heapq

def huffman(freq):
    Q = [(f, i, ch) for i, (ch, f) in enumerate(freq.items())]
    heapq.heapify(Q)                          # Q = C (min-priority queue)
    hitung = len(Q)
    for _ in range(len(freq) - 1):            # n - 1 kali penggabungan
        fx, _, x = heapq.heappop(Q)           # EXTRACT-MIN
        fy, _, y = heapq.heappop(Q)           # EXTRACT-MIN
        heapq.heappush(Q, (fx + fy, hitung, (x, y)))   # INSERT(Q, z)
        hitung += 1
    kode = {}
    def telusuri(simpul, awalan):
        if isinstance(simpul, str):
            kode[simpul] = awalan or "0"
        else:
            telusuri(simpul[0], awalan + "0")    # anak kiri: 0
            telusuri(simpul[1], awalan + "1")    # anak kanan: 1
    telusuri(Q[0][2], "")
    return kode

def decode(bit, kode):
    balik = {v: k for k, v in kode.items()}
    hasil, buf = "", ""
    for b in bit:
        buf += b
        if buf in balik:
            hasil += balik[buf]
            buf = ""
    return hasil

freq = {"a": 45, "b": 13, "c": 12, "d": 16, "e": 9, "f": 5}
kode = huffman(freq)
print(dict(sorted(kode.items())))
print(sum(freq[c] * len(kode[c]) for c in freq), "ribu bit")
print("".join(kode[c] for c in "face"), decode("10111011000", kode))
```

Keluaran program

```
{'a': '0', 'b': '101', 'c': '100', 'd': '111', 'e': '1101', 'f': '1100'}
224 ribu bit
110001001101 beca
```

ANALISIS DAN HUBUNGAN KONSEP

6. Kebenaran dan Running Time

6.1 Mengapa Huffman optimal?

Cormen dkk. (2009, Bab 16.3) membuktikan dua hal yang sesuai dengan dua sifat Greedy dari Modul 13:

1. **Greedy-choice property** (Lemma 16.2): ada pohon kode optimal yang menempatkan dua karakter berfrekuensi terkecil, x dan y, sebagai daun bersaudara pada kedalaman terbesar. Buktinya dengan argumen pertukaran: menukar x dan y dengan dua daun terdalam tidak menambah total bit.
2. **Optimal substructure** (Lemma 16.3): jika x dan y diganti satu karakter baru z dengan frekuensi $x.freq + y.freq$, maka pohon optimal untuk himpunan karakter yang lebih kecil itu, dengan z dipecah kembali menjadi x dan y, adalah pohon optimal untuk himpunan semula.

6.2 Running time

Perulangan berjalan $n - 1$ kali. Jika Q diimplementasikan sebagai *binary min-heap*, setiap EXTRACT-MIN dan INSERT membutuhkan $O(\lg n)$, dan pembentukan heap awal $O(n)$. Total running time: $O(n \lg n)$, dengan n adalah banyaknya jenis karakter (bukan panjang berkas).

6.3 Catatan tentang kode yang tidak tunggal

Jika ada frekuensi yang sama, urutan pengambilan dari Q dapat berbeda, sehingga pohon dan kodenya bisa berbeda. Pertukaran label 0 dan 1 juga menghasilkan kode lain. Semua variasi itu tetap optimal: **total bit**-nya sama, yaitu 224.000 bit pada contoh ini.

6.4 Keterkaitan

Huffman adalah contoh Greedy yang membangun solusi dengan **menggabungkan**, mirip dengan algoritma Kruskal (Modul 20) yang menggabungkan komponen-komponen graf dengan memilih busur paling ringan. Kode Huffman (atau variannya) dipakai sebagai salah satu tahap pada berbagai format kompresi modern.

KESIMPULAN

7. Kesimpulan

- Kode panjang berubah memberi kode pendek pada karakter yang sering muncul; pada contoh, 224.000 bit dibandingkan 300.000 bit (kode tetap) dan 800.000 bit (ASCII).
- Kode prefiks tidak memiliki kode yang menjadi awalan kode lain, sehingga selalu dapat di-decode tanpa ambigu; kode prefiks digambarkan sebagai pohon biner dengan label 0 (kiri) dan 1 (kanan).
- HUFFMAN berulang kali menggabungkan dua simpul berfrekuensi terkecil ($n - 1$ kali) dan berjalan dalam $O(n \lg n)$ dengan min-heap.
- Untuk contoh bahan kuliah: a = 0, b = 101, c = 100, d = 111, e = 1101, f = 1100; `10111011000` di-decode menjadi "beca".
- Kode Huffman optimal karena memenuhi greedy-choice property dan optimal substructure.

Kuis Pemahaman

Kerjakan semua soal terlebih dahulu tanpa melihat kunci jawaban. Kunci jawaban beserta pembahasannya ada di halaman berikutnya.

1 **PILIHAN GANDA**

Manakah yang merupakan kode prefiks?

A. {0, 01, 11}
B. {0, 10, 110, 111}
C. {1, 10, 100}
D. {00, 0, 11}

2 **PERHITUNGAN**

Sebuah berkas berisi 1.000 karakter dari 10 jenis karakter. Berapa bit minimum per karakter untuk kode panjang tetap, dan berapa total bitnya?

3 **PERHITUNGAN**

Dengan kode Huffman contoh modul ini, decode barisan `0101100111`, lalu encode kata "dad".

4 **PENELUSURAN**

Bangun pohon Huffman untuk frekuensi A:20, B:10, C:5, D:40, E:25. Tuliskan urutan penggabungan dan total bit (frekuensi × panjang kode).

5 **BENAR/SALAH DENGAN ALASAN**

"Karena Huffman bersifat greedy, kode yang dihasilkannya tidak selalu optimal." Benar atau salah?

6 **ANALISIS**

Mengapa pohon kode optimal tidak pernah memiliki simpul dalam dengan satu anak saja? Gunakan pohon kode panjang tetap pada modul ini sebagai contoh.

7 **PERHITUNGAN**

Jika ada 256 jenis karakter, berapa kali perulangan HUFFMAN dijalankan? Berapa perkiraan banyaknya operasi heap?

8 **PILIHAN GANDA**

Pada contoh modul ini, karakter mana yang kodenya paling panjang dan mengapa?

A. a, karena muncul paling sering.
B. e dan f, karena frekuensinya paling kecil sehingga digabung paling awal dan berada paling dalam.
C. d, karena berada di urutan terakhir.
D. Semua sama panjang.

Kunci Jawaban dan Pembahasan

Bandingkan jawabanmu, lalu baca pembahasannya untuk memahami alasan di balik setiap jawaban.

Soal 1

Jawaban: B. A bukan kode prefiks (0 adalah awalan 01). C bukan (1 awalan 10). D bukan (0 awalan 00).

Soal 2

Dibutuhkan $\lceil \lg 10 \rceil = 4$ bit, karena $2^3 = 8 < 10 \leq 16 = 2^4$. Total $1.000 \times 4 =$ **4.000 bit**.

Soal 3

`0|101|100|111` = **"abcd"**. Encoding "dad": d = 111, a = 0, d = 111, sehingga `1110111` (7 bit).

Soal 4

Awal: C:5, B:10, A:20, E:25, D:40. (1) C:5 + B:10 = 15. (2) [15] + A:20 = 35. (3) E:25 + [35] = 60. (4) D:40 + [60] = 100. Panjang kode: D = 1, E = 2, A = 3, B = 4, C = 4 (misalnya D = 0, E = 10, A = 111, C = 1100, B = 1101). Total = $40 \cdot 1 + 25 \cdot 2 + 20 \cdot 3 + 10 \cdot 4 + 5 \cdot 4 = 40 + 50 + 60 + 40 + 20 =$ **210**.

Soal 5

Salah. Tidak semua algoritma greedy optimal, tetapi Huffman telah **dibuktikan** optimal di antara semua kode prefiks, karena masalahnya memiliki greedy-choice property dan optimal substructure.

Soal 6

Jika simpul dalam hanya punya satu anak, bit pada busur ke anak tunggal itu tidak membedakan apa pun. Kita bisa menghapus simpul tersebut dan menaikkan anaknya, sehingga semua kode di bawahnya lebih pendek 1 bit dan total bit berkurang. Pada pohon kode tetap, simpul "14" di kanan hanya punya satu anak; jika dihilangkan, e dan f cukup memakai kode 2 bit (10 dan 11), sehingga kode tetap 3 bit itu bukan yang optimal.

Soal 7

Perulangan berjalan $256 - 1 =$ **255 kali**. Setiap putaran: 2 EXTRACT-MIN dan 1 INSERT, total sekitar $3 \times 255 = 765$ operasi heap, masing-masing $O(\lg 256) = O(8)$ langkah perbandingan, ditambah pembentukan heap awal.

Soal 8

Jawaban: B. e (9) dan f (5) adalah dua frekuensi terkecil, sehingga digabung pada loop pertama dan menjadi daun terdalam (kode 4 bit).

Referensi

Sumber utama: Materi 10.2 ALGORITMA GREEDY-ALGORITMA HUFFMAN (bahan kuliah Perancangan dan Analisis Algoritma). Penjelasan tambahan, koreksi, dan pengayaan mengacu pada pustaka berikut.

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). Cambridge, MA: MIT Press. ISBN 978-0-262-03384-8.
2. Huffman, D. A. (1952). A Method for the Construction of Minimum-Redundancy Codes. *Proceedings of the IRE*, 40(9), 1098–1101. https://doi.org/10.1109/JRPROC.1952.273898